



Mergin Maps Community Edition

Run your own sync server, on your laptop

Laax workshop

<https://tinyurl.com/laaxqgis>

Saber Razmjooei

- **Role:** Director
- **Company:** Lutra Consulting Limited
- **Background:** QGIS power user and open source advocate
- **Contact:** LinkedIn

Julien Waddle

- **Role:** Geospatial consultant
- **Company:** Camptocamp
- **Background:** cartographer turned geomatics specialist, discovered open source in 2012 and haven't looked back since
- **Contact:** julien.waddle@camptocamp.com

Camptocamp

Open source geospatial, since 2001.

2001

founded

150+

open source experts

3

countries: CH, FR, DE

- **Geospatial:** GIS and spatial data infrastructures, custom apps, consulting, support, training
- **QGIS Certified Organization** and **GeoServer core contributor**
- **Full open source stack:** **Mergin Maps**, **QField**, geOrchestra, GeoMapFish, GeoServer / GeoServer Cloud, G3W-SUITE, MapServer, Cesium, etc.

Today

MODULE	
00	What is Mergin Maps, what runs where
01	Get the server files, configure
02	Start the stack
03	Init, first login, admin panel
04	QGIS plugin, first project
05	The Mergin Maps app on your laptop, the round trip
06	Command line: server and client CLIs
07	DB Sync: field data into PostGIS
08	Troubleshooting
09	Wrap-up, towards production

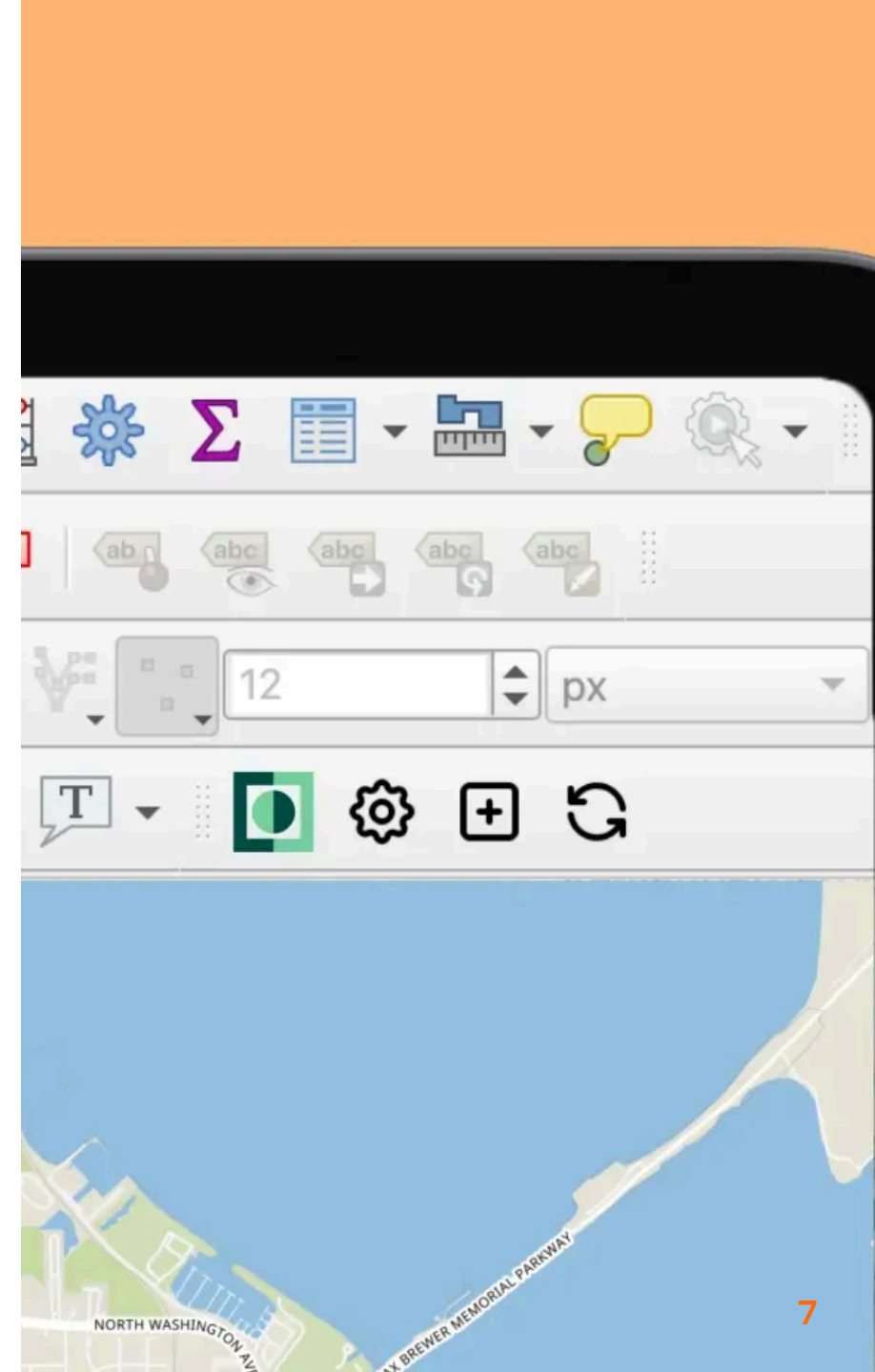
00 What is Merjin Maps



The problem it solves

- Field teams collect data on phones, offline.
- Desktop GIS people design the project and analyse the data in QGIS.
- Both edit the **same** GeoPackage. Without a server, someone loses work.

Mergin Maps stores the project centrally, keeps every version, and merges concurrent edits to GeoPackage layers.

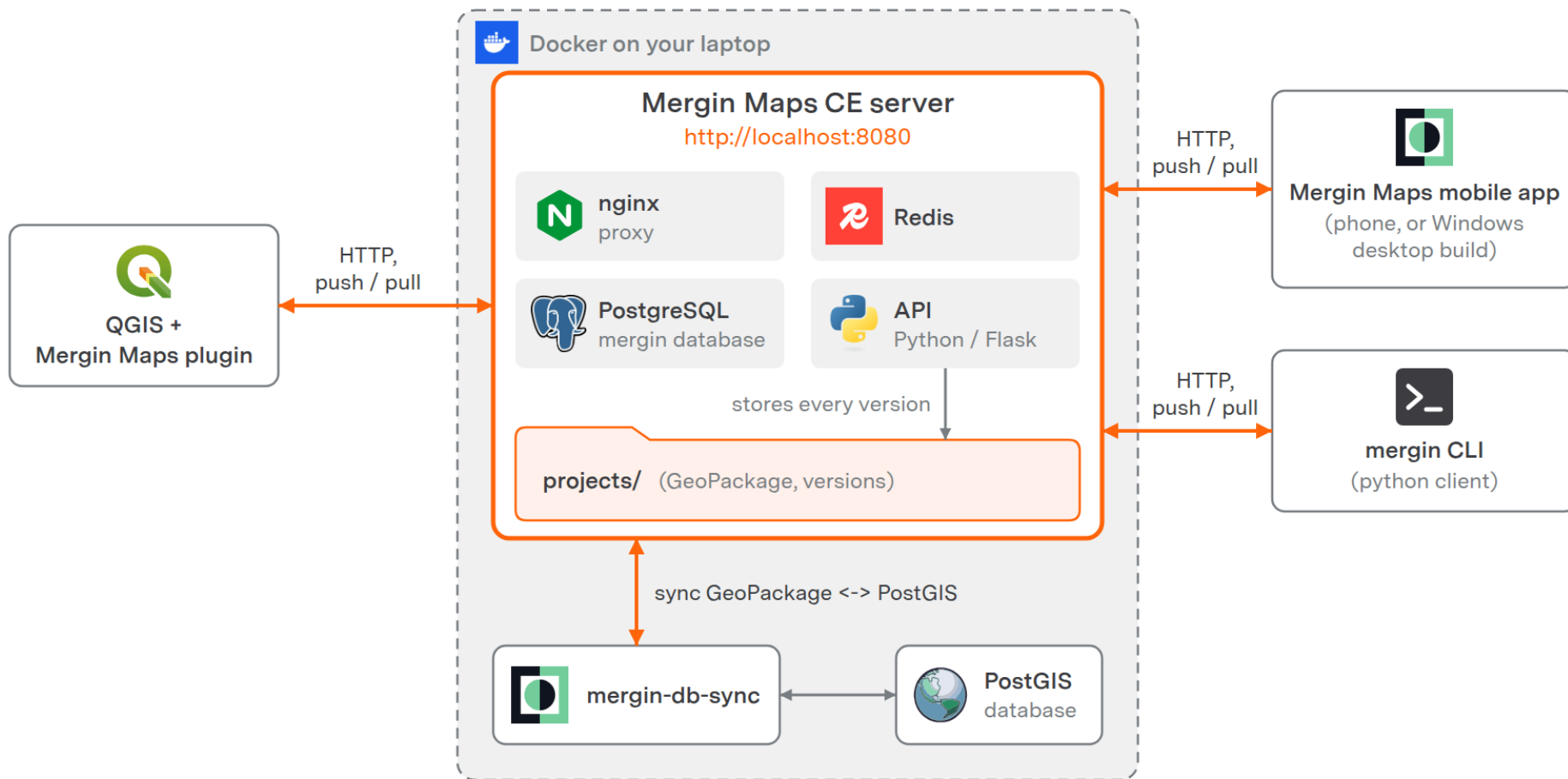


Three pieces

PIECE	RUNS ON	ROLE
Mergin Maps server	a machine you control	stores projects, history, users
QGIS plugin	your desktop	create project, push, pull
Mobile app	phone, tablet, also a Windows desktop build	download, survey offline, sync

Plugin and app talk to the server over HTTP. Nothing else.

How the pieces talk



Community vs Enterprise

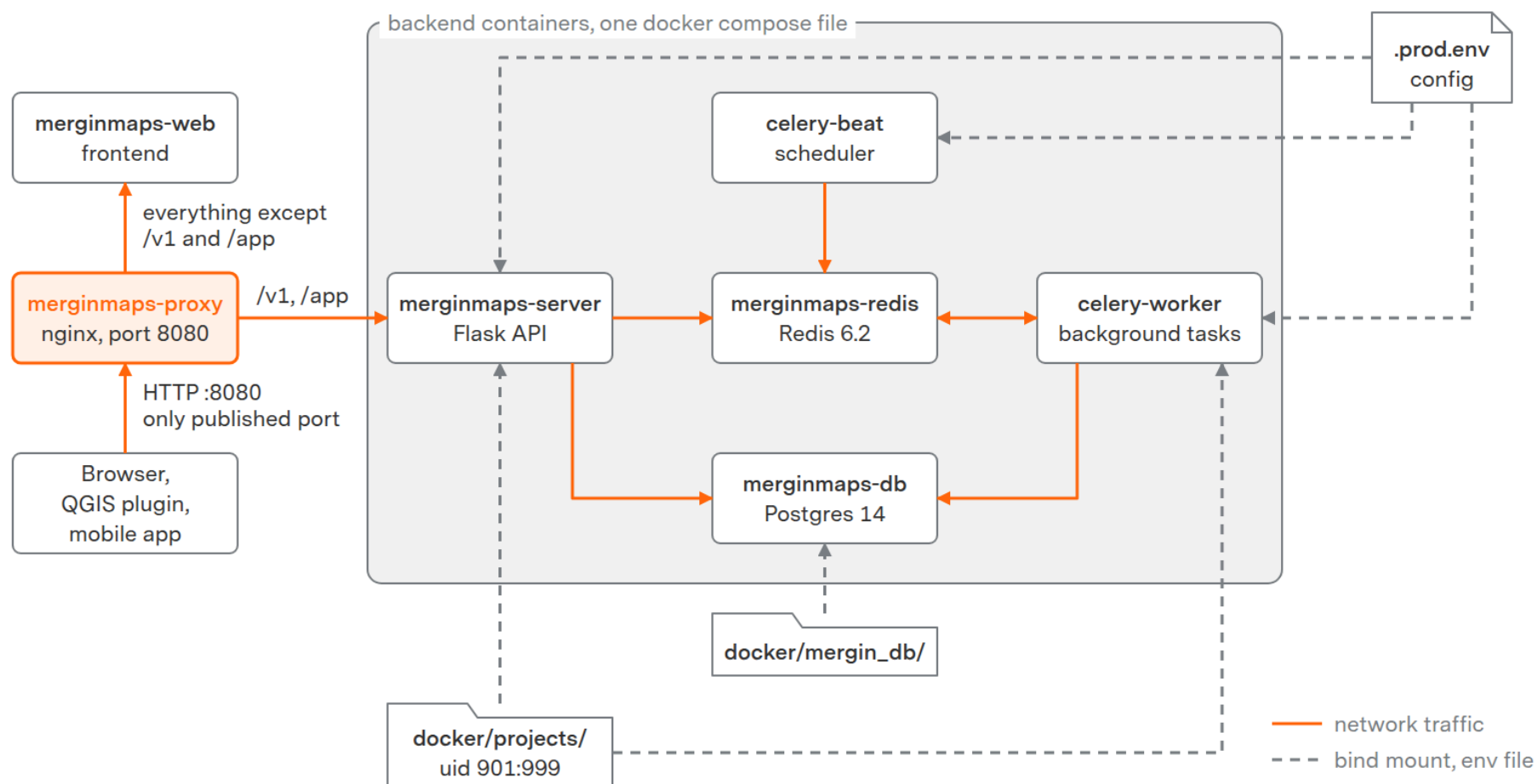
	COMMUNITY EDITION	ENTERPRISE EDITION
Licence	open source, self-hosted	commercial, cloud or on-premises
Workspaces	one, named <code>mergin</code>	many
Web maps, SSO	no	yes
Sync, versions, QGIS + mobile	yes	yes

Today: Community Edition, version 2025.7.3.

What starts when you run `docker compose up`

CONTAINER	IMAGE	JOB
<code>merginmaps-db</code>	<code>postgres:14</code>	users, projects metadata, versions
<code>merginmaps-redis</code>	<code>redis:6.2</code>	queue for background jobs
<code>merginmaps-server</code>	<code>merginmaps-backend</code>	the API, Flask + gunicorn
<code>celery-worker</code>	<code>merginmaps-backend</code>	background jobs: mails, cleanup
<code>celery-beat</code>	<code>merginmaps-backend</code>	scheduler for those jobs
<code>merginmaps-web</code>	<code>merginmaps-frontend</code>	the web UI
<code>merginmaps-proxy</code>	<code>nginx-unprivileged</code>	single entry point, port 8080

Who talks to whom



Where your data lives

```
docker/  
  mergin_db/      Postgres data files  
  projects/       every project, every file version  
  diagnostic_logs/ server diagnostics  
  .prod.env       all configuration
```

- bind-mounted host folders: remove the containers, the data stays
-  **back these up in production**
-  server and worker run as user **901**, group **999**: those folders must belong to them

One request, end to end

1. QGIS or the app calls `http://localhost:8080`
 2. the **proxy** sends API calls to the **server**, the rest to the **web UI**
 3. the **server** saves files in `projects/`, metadata in **Postgres**
 4. slow jobs go through **redis** to the **celery worker**
- one port: **8080**
- ⚠ the server must know its own address: `MERGIN_BASE_URL`, today `http://localhost:8080`

01 Get the server files

Windows only: where to work

Everything happens inside the **Ubuntu (WSL)** terminal.

- Work in `~/workshop_mm_laax` inside your WSL home, never in `/mnt/c/...`. Bind mounts and file ownership behave like Linux there.
- Docker Desktop, Settings, Resources, WSL integration: your Ubuntu distribution must be ticked.
- `docker ps` in the WSL terminal must work before you continue.

One folder for the day: `~/workshop_mm_laax`

```
mkdir -p ~/workshop_mm_laax && cd ~/workshop_mm_laax
git clone https://github.com/julsbreakdown/laax_mm_ws.git laax
cd laax
ls
```

```
common/  dbsync/  docker/  workshop/
```

Repo: <https://tinyurl.com/laaxqgis>. No git? Code, **Download ZIP**, then `unzip`

```
~/Downloads/laax_mm_ws-master.zip && mv laax_mm_ws-master laax.
```

`docker/` and `common/` are copies of `deployment/community` and `deployment/common` from <https://github.com/MerginMaps/server>, plus a container dashboard in `common/telemetry/`. Every command today runs from `~/workshop_mm_laax/laax/docker`.

Create your configuration

```
cd docker  
cp .env.template .prod.env
```

`.prod.env` is read by the server and the two celery containers. Whatever you set there wins over the image's defaults.

Six lines to change.

1. The server's own address

Every client today runs on this laptop, so:

```
MERGIN_BASE_URL=http://localhost:8080
```

Windows: Docker Desktop publishes the port on Windows `localhost` too, so QGIS and the desktop app on Windows reach the server in WSL with the same URL.

If you want to use your phone on your own network:

→ Info is in `ipconfig` or `ifconfig`

2. Secrets

```
openssl rand -hex 32  
openssl rand -hex 32
```

```
SECRET_KEY=<first value>  
SECURITY_PASSWORD_SALT=<second value>
```

Never leave `fixme`. The server signs sessions and password hashes with these.

3. Mail, database, contact

No SMTP server today, so:

```
MAIL_SUPPRESS_SEND=1  
CONTACT_EMAIL=you@example.com
```

Use the `mergin` database that compose creates, not the Postgres maintenance database.
Uncomment:

```
DB_DATABASE=mergin
```

Checkpoint 01

```
grep -E '^(MERGIN_BASE_URL|SECRET_KEY|SECURITY_PASSWORD_SALT|MAIL_SUPPRESS_SEND|CONTACT_EMAIL|DB_DATABASE)= ' .prod.env
```

Six lines, none containing `fixme`, `MERGIN_BASE_URL` equal to `http://localhost:8080`.

02 Start the stack

Data folders first

```
cd ~/workshop_mm_laax/laax/docker  
sh ../common/set_permissions.sh projects  
sh ../common/set_permissions.sh diagnostic_logs
```

The script asks for `sudo`. It creates the folder and gives it to **901:999**, the user the server runs as.

Why before `up`: if Docker creates a missing bind-mount folder itself, it belongs to root and the server cannot write.

Up

```
docker compose up -d
```

First run pulls about 1.5 GB of images. Then:

```
docker compose ps
```

Seven containers, all `Up`. Redis and Postgres may show `Up` a few seconds before the server does.

What just happened

`docker compose` read `docker-compose.yml` in the current folder:

- created a network `mergin` and seven containers
- mounted `./projects`, `./mergin_db`, `../common/nginx.conf` into them
- passed `.prod.env` to server, celery-worker and celery-beat
- published the proxy on port 8080, plus 8082 which nothing listens on today

The compose project is named after the folder, `docker`. Run compose commands from this folder, always.

Logs

```
docker compose logs -f server celery-worker
```

Ctrl+C leaves the containers running. Look for:

```
[INFO] Listening at: http://0.0.0.0:5000  
celery@... ready.
```

Errors about relation "user" does not exist are expected until module 03.

The server describes itself

```
curl -s http://localhost:8080/ping | python3 -m json.tool
```

Every client reads this first. Worth a look:

KEY	MEANING
<code>blacklist_dirs</code> , <code>blacklist_files</code>	never uploaded: <code>.mergin</code> , <code>.DS_Store</code> , <code>.directory</code>
<code>endpoints</code>	the whole API. A push is <code>push</code> , then <code>push/chunk</code> per piece, then <code>push/finish</code>
<code>maintenance</code>	<code>true</code> freezes writes, for upgrades and backups
<code>version</code>	<code>2025.6.2</code> , what image <code>2025.7.3</code> reports

Checkpoint 02

```
docker compose ps  
curl -s http://localhost:8080/ping | head -c 80; echo
```

Seven `Up`, and `/ping` answers with JSON starting `{"base_url": "v1"}`.

Windows: also open <http://localhost:8080> in a browser. Accept the firewall prompt if Docker Desktop shows one.

03 Init, first login, admin panel

Create the schema and the admin

```
docker compose exec server flask init -e owner@example.com
```

`flask` is the server's own command line tool, it runs inside the `server` container:

1. creates all tables in the `mergin` database
2. creates the admin account, username **owner**, random 12-character password, printed once
3. runs the server health check

```
Admin user created. Please save generated password.  
Email: owner@example.com  
Username: owner  
Password: kY3...
```

Copy the password now. The username is the email's local part. `admin` is a reserved word, `admin@example.com` would give `admin0`.

No password printed? The database is not empty. `docker compose down -v` keeps `mergin_db/` ; wipe

Health check

```
docker compose exec server flask server check
```

```
Base URL of server is http://localhost:8080  
Database initialized properly  
Permissions granted for /data folder.  
Celery is running properly
```

Mergin Maps version: 2025.6.2 is what image 2025.7.3 reports. Expected.

A red permissions line means the folders were created by Docker, not by `set_permissions.sh`. Fix:

```
sh ../common/set_permissions.sh projects  
docker compose restart server celery-worker
```

Log in

Open **<http://localhost:8080>** in the browser. Same URL QGIS and the field device will use.

Username `owner`, the generated password.

You land in the workspace **mergin**, the single Community Edition workspace.

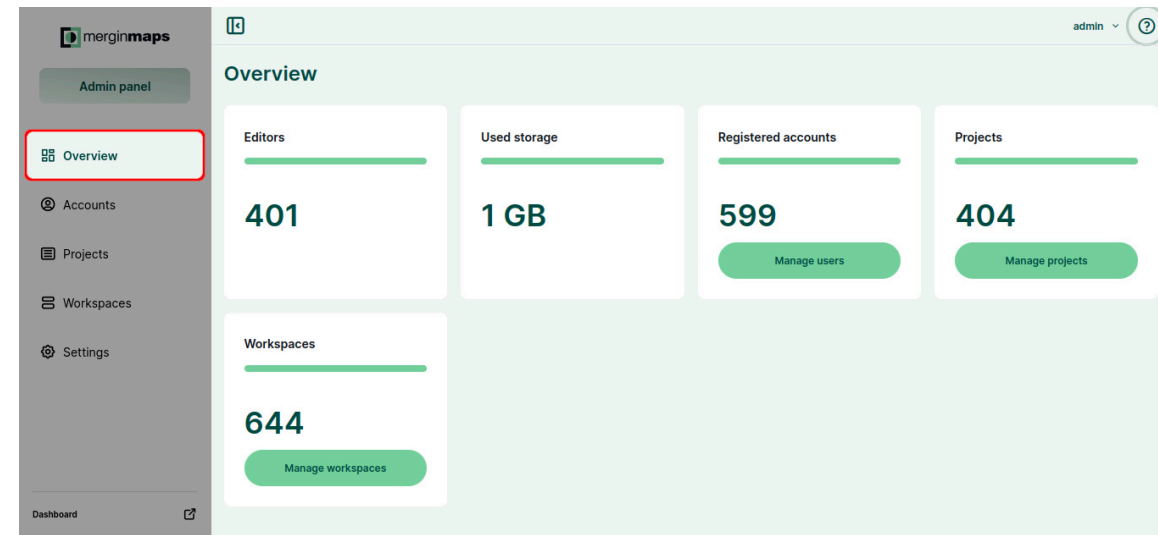
Empty projects list.



Admin panel tour

Top right menu, **Admin panel**.

TAB	WHAT YOU SEE
Overview	accounts, projects, storage used
Accounts	every user, click one to make admin, deactivate, delete
Projects	every project, download, open dashboard
Settings	update checks, usage report





Create the field user

Accounts, **Add user.**

- username `field`
- your email again
- a password you will type again in module 05

With `MAIL_SUPPRESS_SEND=1` no invitation is sent. The password you type here is the one.

Create user



First name

Email

Password 

Enter password



Cancel

Create

Checkpoint 03

Admin panel, Accounts: two rows, `owner` and `field`.

```
docker compose exec server flask server check
```

No red line.

04 QGIS plugin, first project

Install the plugin

QGIS, Plugins, **Manage and Install Plugins**, search `Mergin Maps` , install.

No internet: Plugins, **Install from ZIP**.

A Mergin Maps toolbar and a Mergin Maps entry in the Browser panel appear.



Point it at your server

Toolbar, **Configure Mergin Maps** plugin.

- tick **Custom Mergin Maps server**
- URL: `http://localhost:8080`
- username `owner` , the generated password
- **Test connection**, then OK

Tip: QGIS, Settings, User Profiles, **New profile**, name it `workshop` . Plugin settings are per profile, so your everyday QGIS stays untouched and a cloud login never gets in the way.

Merjin Settings

merginmaps

sarah

.....

☒ Save credentials

☒ Custom Merjin Maps server

Test Connection

Don't have an account yet? [Sign up](#) now!

OK Cancel

A small survey project

New QGIS project, save it as `~/workshop_mm_laax/laax-survey/laax-survey.qgz`.

Layer, Create Layer, **New GeoPackage Layer**:

- file `~/workshop_mm_laax/laax-survey/data.gpkg`, table `observations`, geometry Point, CRS EPSG:2056
- fields: `species` text, `count` integer, `note` text, `photo` text

Add an OpenStreetMap XYZ background from the Browser panel.

Short on time: `workshop/qgis/laax-survey/` holds this project ready, form included. Copy it to `~/workshop_mm_laax/laax-survey`.

A form worth filling on a phone

Layer properties, Attributes Form, drag and drop designer:

- `species` : Value Map, a handful of entries
- `count` : Range, 1 to 100
- `photo` : Attachment, relative path

Save the project.



Push it

Browser panel, Mergin Maps, right click **Create new project**, from current QGIS project.

Name `laax-survey` . Upload.

Then in the web UI: project `laax-survey` , version **v1**,
two files: `laax-survey.qgz` and `data.gpkg` .

 Create new Mergin Maps project ✕

New basic QGIS project

Recommended

The easiest way to start from scratch, not requiring an existing QGIS project. This option will create and upload a project with a basemap and a survey layer will be created, ready for further customization.

Package current QGIS project

Recommended

Use this option if you have an existing QGIS project which has not yet been updated to be used offline or in the cloud. We will make a copy of your project and take care of packaging of map layers so that all data are contained in a single folder.

Use current QGIS project as is

Use this option if you already have a QGIS project ready for offline or cloud use (i.e. QGIS project file and all data sources are contained in a single folder). Data are uploaded without any extra packaging steps.

< Back

Next >

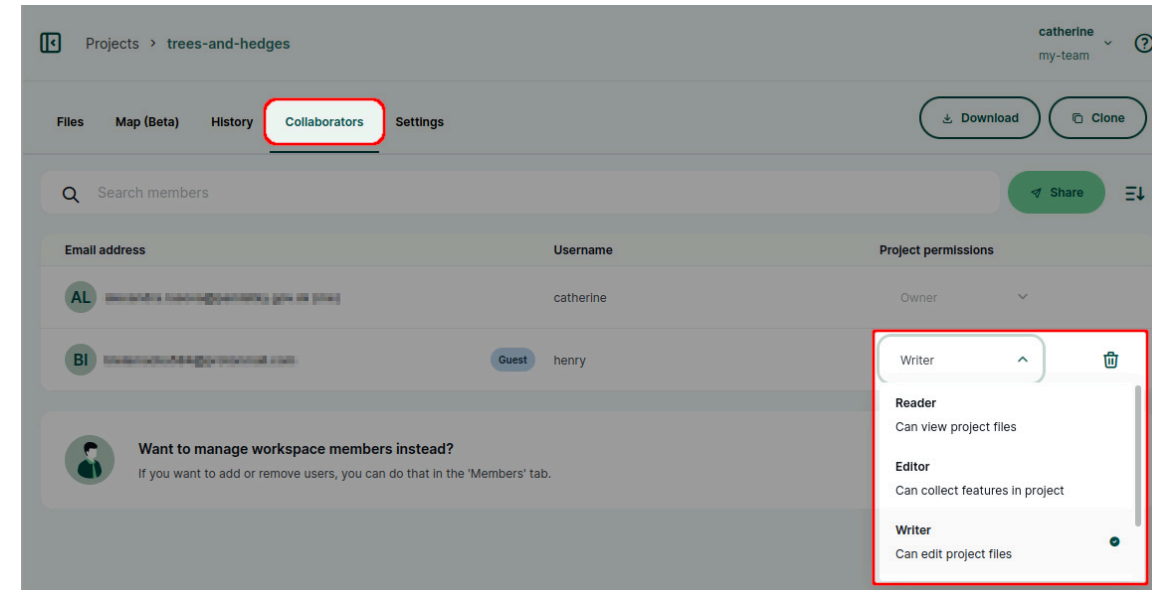
Cancel



Share it with the field user

Web UI, project `laax-survey`, **Collaborators**, add field as **Editor**.

Community Edition default: `GLOBAL_READ=False`, users see only projects shared with them.



Checkpoint 04

Web UI, project `laax-survey` :

- version v1
- collaborators: `owner` as owner, `field` as editor

QGIS Browser panel shows the project with a synced icon.

05 The field device aka Phone App

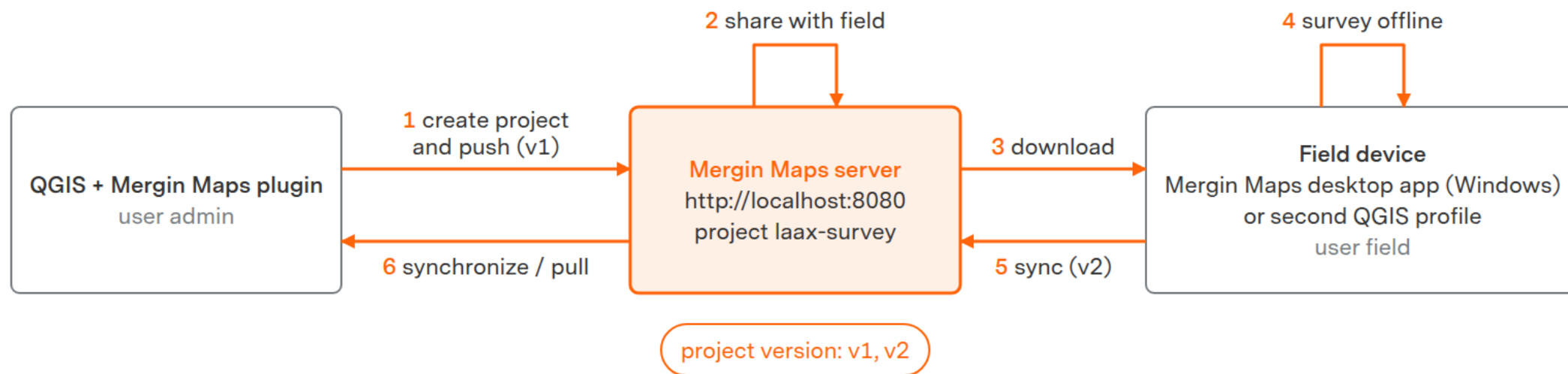
The field device "lives" on your laptop today.

Two ways to play the field device

	WINDOWS	LINUX
Field device	Mergin Maps desktop app, <code>merginmaps-2026-4-0-win64.exe</code>	Mergin Maps Linux build from the trainer's stick
Same UI as the phone	yes	yes
Server URL	<code>http://localhost:8080</code>	<code>http://localhost:8080</code>
Logs in as	<code>field</code>	<code>field</code>



The round trip in six steps



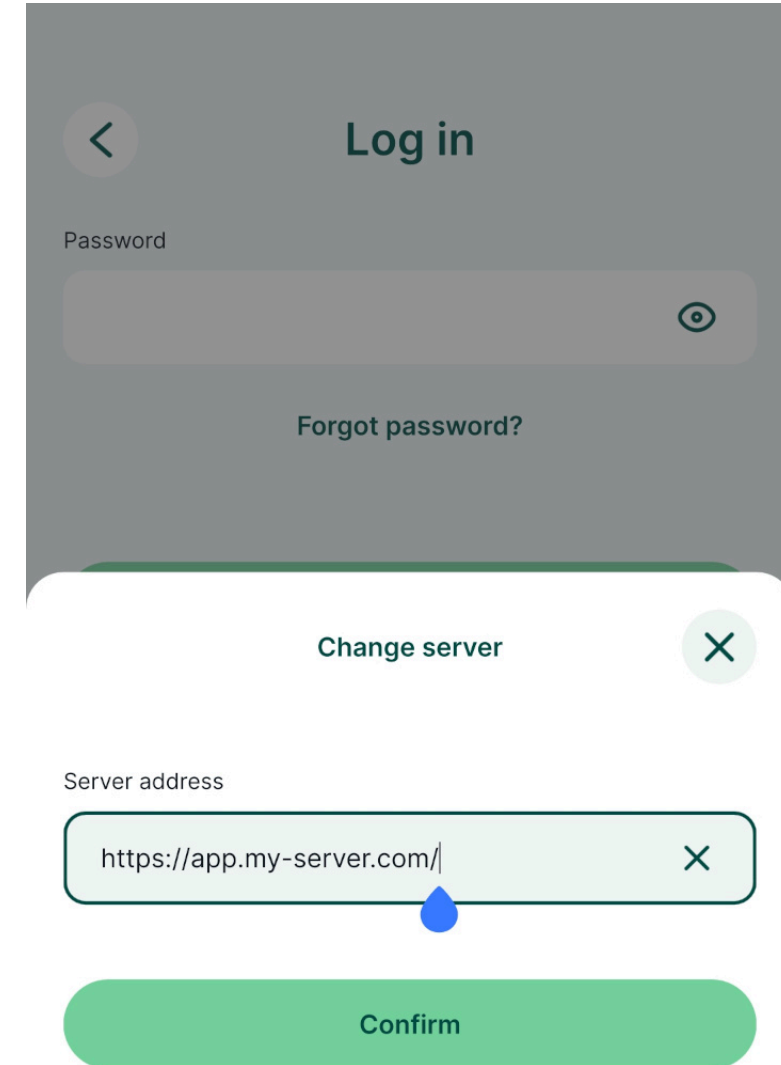


Windows: install and point the app

Installer: <https://github.com/MerginMaps/mobile/releases>,
merginmaps-2026-4-0-win64.exe .

Start Mergin Maps, signed out, login screen. Click the **server address at the bottom**.

Enter `http://localhost:8080` , Confirm. Log in as `field` .



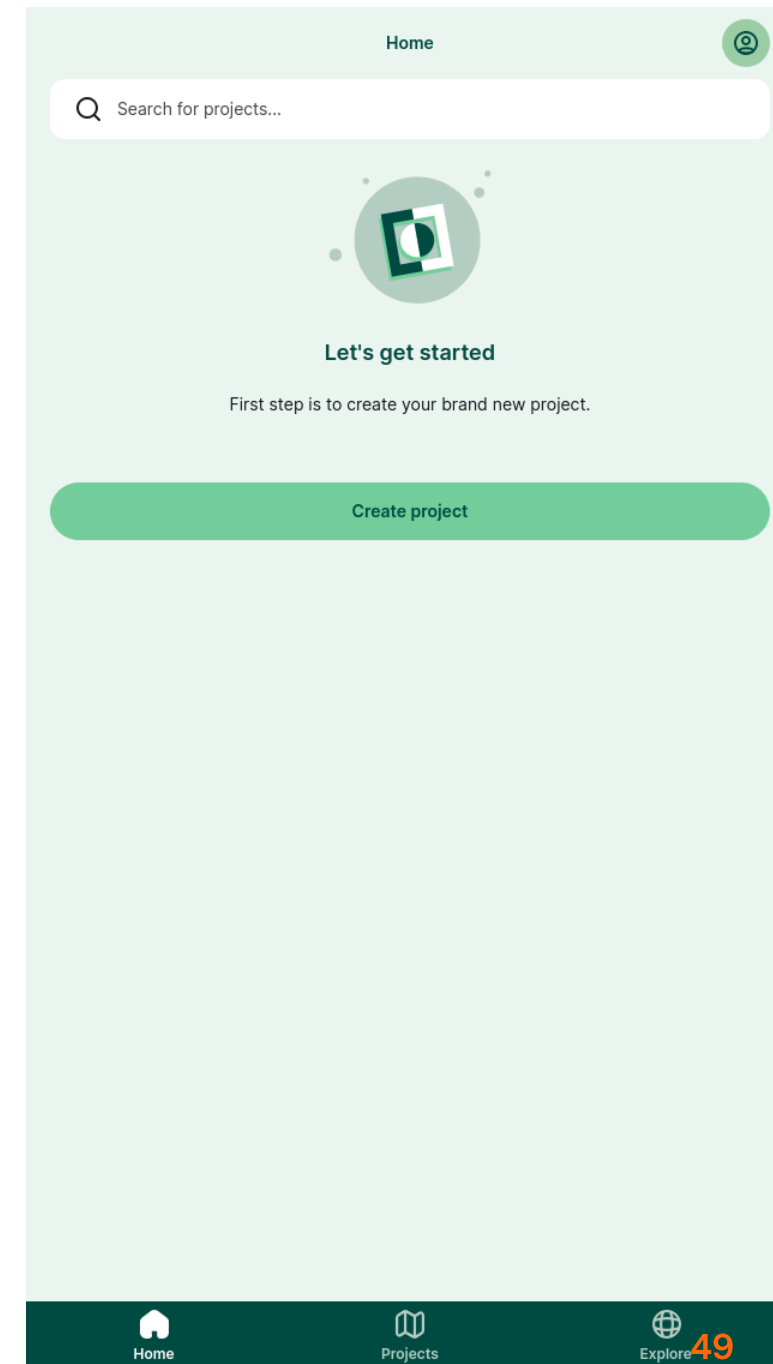


Linux: install the app

```
mkdir -p ~/workshop_mm_laax/merginmaps && cd ~/workshop_mm_laax/merginmaps
unzip ~/Downloads/Mergin\ Maps\ *\ x86_64.zip
tar -xzf input-*.tar.gz && rm input-*.tar.gz
cp ~/workshop_mm_laax/laax/workshop/linux-app/run.sh . && ./run.sh
```

Not published in the releases, built by the project's Linux CI.
`run.sh` sets library paths, PROJ, and links the app's fixed data folder (the path of the machine that built it) to `~/workshop_mm_laax/field`, one `sudo` prompt the first time.

Then exactly as on Windows: profile icon, Log in, server address at the bottom, `http://localhost:8080`, log in as `field`.



App: download and survey

Projects, **Shared with me**, `laax-survey`, download.

Open it. Record, click the map, fill the form, attach a photo from disk, save.

Add two or three points. Then back on the project list, **Sync**.



Back in the owner window

Web UI, project `laax-survey`, version **v2**, History tab: changed by `field`, `data.gpkg` modified.

QGIS owner window, Browser panel, `laax-survey`, right click, **Synchronize**. The points appear.

The screenshot shows the merginmaps web interface. On the left is a sidebar with navigation links: Dashboard, Projects (highlighted with a red box), Members, Subscriptions, and Settings. The main area displays the 'History' tab for the project 'my-survey'. At the top of the main area, there are tabs for Files, Map, History (highlighted with a red box), Collaborators, and Settings. To the right of these tabs are 'Download' and 'Clone' buttons. Below the tabs is a table with the following data:

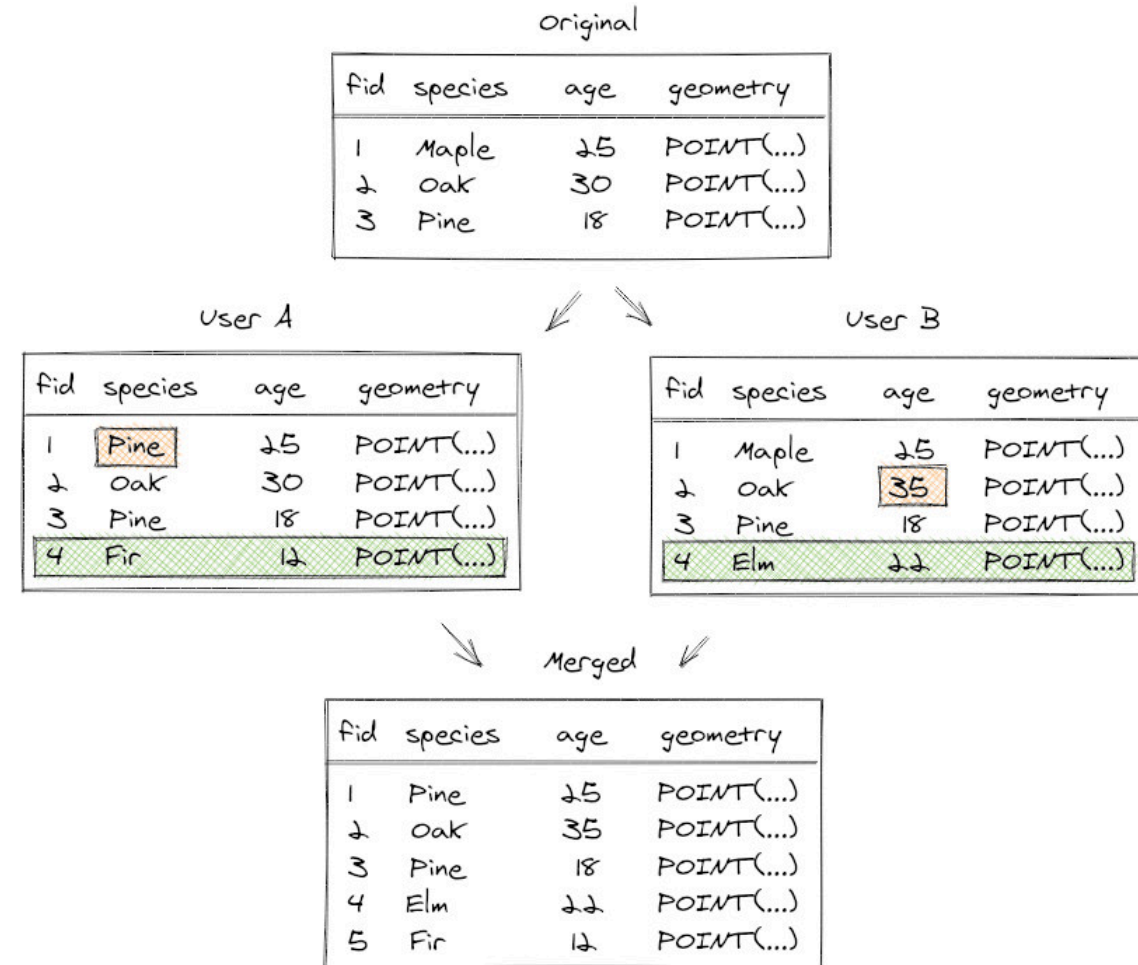
Version	Created	Author	Files added	Files edited	Files removed	Size	
v9	0 minutes ago	catherine	1	3	0	11.74 MB	
v8	6 minutes ago	catherine	0	3	0	4.61 MB	
v7	11 minutes ago	catherine	0	3	0	4.61 MB	

Concurrent edits

- Owner window: move one point, change a note, synchronize.
- Field device at the same time: add another point, sync.

No conflict. The server merges GeoPackage changes row by row.

Edits to the **same row** from both sides: last sync wins, the history keeps both.



With a real phone, later

Same steps, two differences:

- `MERGIN_BASE_URL` and the URL typed on the phone are the laptop's LAN address, not `localhost`
- the laptop firewall must accept port 8080, see deck 08

Checkpoint 05

- Web UI: `laax-survey` at version v3 or higher, history shows both `owner` and `field`
- Owner QGIS window: points created by the field device visible in `observations`
- Field device: point moved by owner is at its new place

The round trip is done. Everything else is operations.

06 Command line

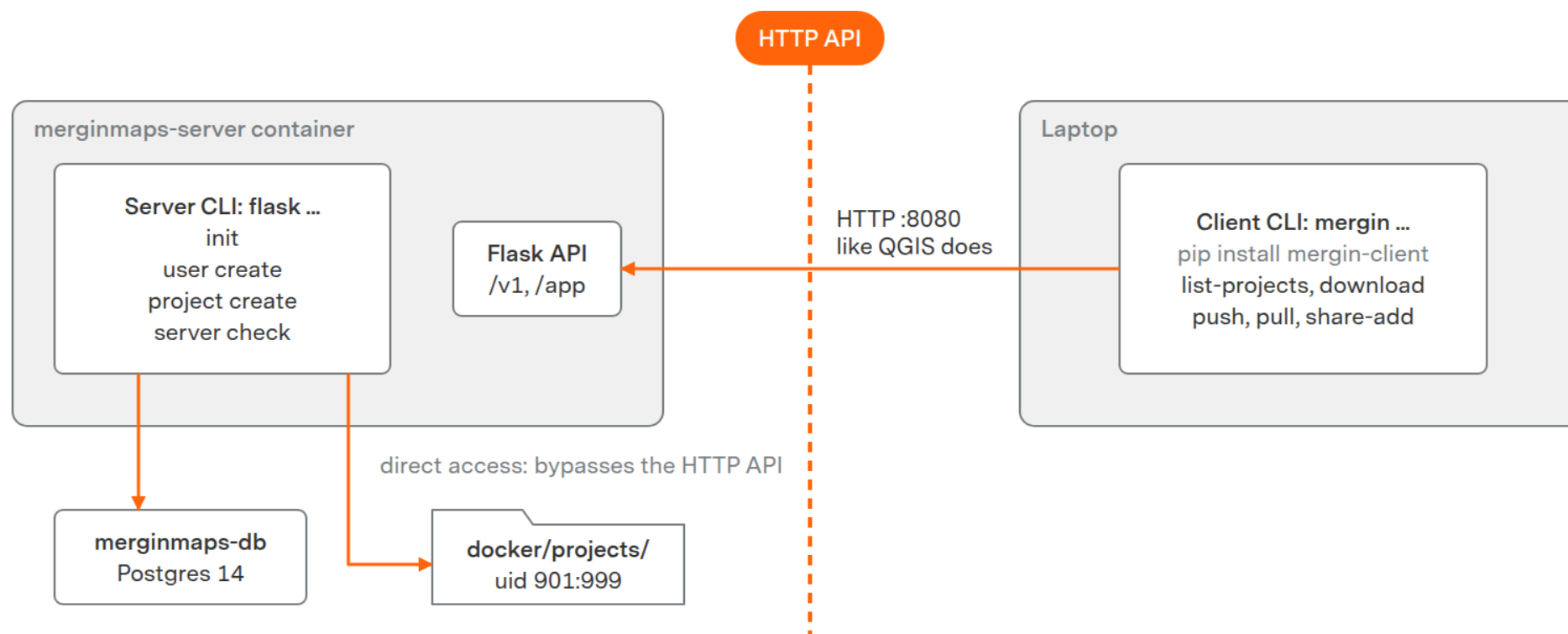
Everything you clicked, scripted.

Two different CLIs

	SERVER CLI, <code>FLASK ...</code>	CLIENT CLI, <code>MERGIN ...</code>
Runs	inside <code>merginmaps-server</code>	on any machine with Python
Talks to	the database and <code>projects/</code> directly	the HTTP API, like QGIS and the app
Who	the server administrator	any user, with their own login
Typical job	init, create users, check health	push, pull, share, list projects



Where each one enters



Server CLI: what exists

```
docker compose exec server flask --app application --help
```

COMMAND	JOB
<code>init -e EMAIL</code>	create schema and admin, health check
<code>user create USER PASS --email E [--is-admin]</code>	create an account
<code>project create NAME WORKSPACE OWNER</code>	blank project
<code>project download WORKSPACE/NAME --version N --directory DIR</code>	files of one version
<code>project remove WORKSPACE/NAME</code>	delete a project
<code>server check , server permissions --path P</code>	diagnostics
<code>db upgrade</code>	schema migration on upgrade

`flask <group> <command> --help` on any of them.

Batch-create users from a CSV

workshop/users.csv :

```
username,password,email,role
DollarEsri,Workshop2026!,dollaresri@example.com,user
ShapefileMustDie,Workshop2026!,shapefilemustdie@example.com,user
ByeByeFME,Workshop2026!,byebyeefme@example.com,user
QgisForever,Workshop2026!,qgisforever@example.com,admin
```

One shell loop:

```
tail -n +2 ../workshop/users.csv | while IFS=, read -r u p e role; do
    docker compose exec -T server flask user create "$u" "$p" --email "$e" \
        $([ "$role" = admin ] && echo --is-admin) </dev/null
done
```

`</dev/null` matters: without it `docker compose exec` eats the rest of the CSV and the loop stops after the first user.

Same idea, projects

One blank project per team, owned by `owner` :

```
for t in north south east; do
  docker compose exec -T server flask project create "survey-$t" mergin owner
done
```

Community Edition workspace is always `mergin` .

Keep a copy of a version outside the server:

```
docker compose exec server flask project download mergin/laax-survey --version 2 --directory /tmp/v2
docker compose cp server:/tmp/v2 ./laax-survey-v2
```

Client CLI: install and log in

On your laptop, not in a container:

```
python3 -m venv ~/workshop_mm_laax/venv && source ~/workshop_mm_laax/venv/bin/activate
pip install mergin-client

export MERGIN_URL=http://localhost:8080
export MERGIN_USERNAME=owner
export MERGIN_PASSWORD=<owner password>
mergin list-projects mergin
```

Windows: run this in the WSL terminal as well. `MERGIN_URL` is read by every command.

Client CLI: the round trip, scripted

```
mergin download mergin/laax-survey ./laax-survey  
cd laax-survey  
mergin status  
mergin pull  
mergin push
```

Create from a folder in one go:

```
mergin create mergin/road-inventory --from-dir ./road-inventory
```

Share with everyone from the CSV

```
mergin share-add mergin/laax-survey DollarEsri ShapefileMustDie ByeByeFME --permissions writer  
mergin share mergin/laax-survey
```

Or read the names from the file:

```
mergin share-add mergin/laax-survey $(tail -n +2 workshop/users.csv | cut -d, -f1) --permissions writer
```

When to use which

- New field campaign, 30 surveyors: the CSV loop, then `mergin share-add`.
- Nightly export of the latest data: `mergin download` in a cron job.
- Server upgrade: `flask db upgrade`, `flask server check`.
- Something looks broken: `flask server check`, `flask server permissions`.
- Anything about a single user's data: the web UI is faster.

Checkpoint 06

```
mergin list-projects mergin  
mergin share mergin/laax-survey
```

Admin panel, Accounts: DollarEsri, ShapefileMustDie, ByeByeFME, QgisForever present, QgisForever flagged admin. share lists them on laax-survey as writers.

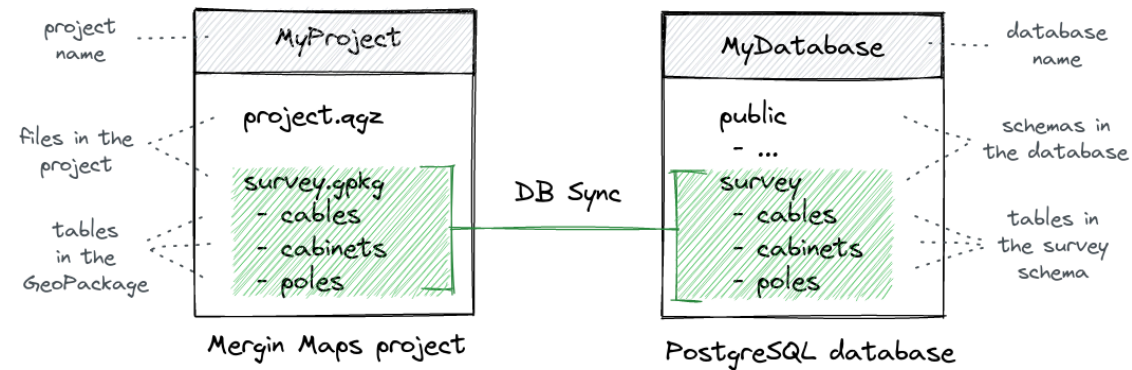
07 DB Sync

Field data lands in PostGIS, and back.

What db-sync is, when to use it

A small daemon. One GeoPackage of a project and one PostGIS schema, kept identical both ways. Every table in the file is synced.

- field points in PostGIS, for SQL and analysis
- dashboards and web maps read PostGIS
- the enterprise database stays the reference
- office edits in PostGIS, surveyors in the app



How it works

PIECE	ROLE
<code>survey</code> schema	the modified schema. You read and edit here
<code>survey_base</code> schema	copy of the last project version seen. Never touch it
geodiff	computes row changes, the same engine as project sync
working copy	the project, downloaded inside the container

Every `sleep_time` seconds:

1. **pull:** new version on the server? apply its diff to `survey` and `survey_base`
2. **push:** `survey` differs from `survey_base` ? upload the diff as a new version

Architecture

CONTAINER	IMAGE	NETWORK	ADDRESS
laax-postgis	postgis/postgis:16-3.4	mergin	laax-postgis:5432 inside, localhost:5433 from the laptop
laax-dbsync	lutraconsulting/mergin-db-sync:2.3.0	mergin	no port, calls the server
merginmaps-proxy	from module 02	mergin	merginmaps-proxy:8080 inside, localhost:8080 from the laptop

`~/workshop_mm_laax/laax/dbsync/docker-compose.yml` joins the network `mergin` as `external`. Its own PostGIS, never `merginmaps-db`.

Before you start:

- server stack up, `mergin/laax-survey` with `data.gpkg` pushed (04), `field` editor and field device ready (05)
- schemas `survey` and `survey_base` do **not** exist. Init creates them

config.yaml

```
cd ~/workshop_mm_laax/laax/dbsync  
cp config.yaml.template config.yaml
```

```
mergin:  
  url: http://merginmaps-proxy:8080  
  username: owner  
  password: <owner password>  
init_from: gpkg  
connections:  
  - driver: postgres  
    conn_info: "host=laax-postgis port=5432 dbname=dbsync user=dbsync password=dbsync"  
    modified: survey  
    base: survey_base  
    mergin_project: mergin/laax-survey  
    sync_file: data.gpkg  
daemon:  
  sleep_time: 10
```

config.yaml, line by line

KEY	MEANING
<code>mergin.url</code>	server as seen from the container . <code>localhost</code> there is the container itself
<code>username</code> , <code>password</code>	a Mergin account with write access to the project
<code>init_from: gpkg</code>	GeoPackage exists, create the schemas. <code>db</code> is the reverse: schema exists, create the GeoPackage
<code>driver</code>	<code>postgres</code> , the only one
<code>conn_info</code>	libpq string, <code>laax-postgis:5432</code> on the docker network
<code>modified</code> , <code>base</code>	the two schemas, both created by init
<code>mergin_project</code>	<code>workspace/project</code> , CE workspace is <code>mergin</code>
<code>sync_file</code>	path of the GeoPackage inside the project
<code>sleep_time</code>	seconds between two passes

Secrets out of the file

The config is read by Dynaconf, so any value can come from the environment.

```
mergin:  
  password: "@format {env[MERGIN_PASSWORD]}"  
connections:  
  - conn_info: "@format host=laax-postgis port=5432 dbname=dbsync user=dbsync password={env[PGPASSWORD]}"
```

Or leave the file as it is and override a key with a double underscore: `MERGIN__PASSWORD=...` in the container's environment.

`docker compose run --rm dbsync /config/config.yaml --show-config` prints the merged result. Pass the variables with `environment:` in the compose file, never commit `config.yaml`.

Start

```
docker compose up -d postgis  
docker compose run --rm dbsync /config/config.yaml --single-run
```

Expected, among the last lines:

```
The base and modified schemas do not exist yet, going to initialize them ...  
Init done!  
No changes on Merjin Maps.  
No changes in the database.  
Push done!
```

Then the daemon:

```
docker compose up -d dbsync  
docker compose logs -f dbsync
```

Going to sleep every 10 seconds. Ctrl+C leaves it running.

Verify in PostGIS

```
docker compose exec postgis psql -U dbsync -d dbsync -c '\dt survey.*'
```

Schema	Name	Type	Owner
survey	observations	table	dbsync

```
docker compose exec postgis psql -U dbsync -d dbsync \  
-c 'select fid, species, count, note from survey.observations'
```

Same rows as the GeoPackage. `geom` is `geometry(Point,2056)`, `fid` the primary key.

Edit in PostGIS, new version on the server

```
docker compose exec postgis psql -U dbsync -d dbsync -c "  
  insert into survey.observations (species, count, note, geom)  
  values ('Marmot', 4, 'from psql',  
         ST_SetSRID(ST_MakePoint(2737500, 1189000), 2056))"
```

Within 10 seconds, in the logs:

```
Pushed new version to Mergin Maps: v4
```

Web UI, `laax-survey`, History: new version by `owner`, `data.gpkg` modified. QGIS, Synchronize: the marmot is on the map.

Edit in QGIS, row in PostGIS

The field device or the owner QGIS window: add a point in `observations`, save, **Synchronize**.

Within 10 seconds:

```
Pulled new version from Mergin Maps: v5
```

```
docker compose exec postgis psql -U dbsync -d dbsync \  
-c 'select count(*) from survey.observations'
```

One more row. Edits and deletes travel the same way.

Look at it from QGIS

Browser panel, PostgreSQL, **New Connection:**

FIELD	VALUE
Host	localhost
Port	5433
Database	dbsync
User, password	dbsync , dbsync

Open `survey.observations` . Edit it here like any PostGIS layer, db-sync pushes your changes.

Never edit `survey_base` .

When db-sync fails

SYMPTOM	CAUSE	FIX
Unable to log in ... localhost:8080 ... Connection refused	url set to localhost	http://merginmaps-proxy:8080
The 'modified' schema exists but the base schema is missing	schema created by hand before init	drop it, or --force-init
Config file config.yaml does not exist.	run --rm dbsync --single-run without the path	put /config/config.yaml first
cp: cannot create regular file 'config.yaml/config.yaml.template'	docker compose up -d ran before the file existed, Docker created a root-owned folder	sudo rm -rf config.yaml , then cp again
--force-init fails, Device or resource busy: '/tmp/dbsync'	a volume mounted exactly on '/tmp/dbsync'	mount the volume on /tmp , as in our compose file

Schema changed in the GeoPackage (new field)? Stop the daemon, one pass with --force-init , start again. Unsynced PostGIS edits are lost.

Checkpoint 07

- a point added with `psql` shows up in QGIS after Synchronize
- a point added on the field device shows up in PostGIS:

```
docker compose exec postgis psql -U dbsync -d dbsync \  
-c 'select count(*) from survey.observations'
```

08 Troubleshooting

Everything on these slides happened for real while building this workshop.

Start problems

SYMPTOM	CAUSE	FIX
error mounting .../common/nginx.conf ... not a directory	common/ missing, Docker created nginx.conf as an empty folder	remove that folder, restore common/ from the repo
server logs: permission denied on /data	projects/ owned by root	sh ../common/set_permissions.sh projects , then docker compose restart server celery-worker
env file .prod.env not found	module 01 skipped	cp .env.template .prod.env
port is already allocated on 8080	another service on 8080	stop it, or change the proxy port in compose and in MERGIN_BASE_URL
HTTP 502 after recreating the server container	nginx resolved the old server address at startup	docker compose restart proxy

Redis and container names

SYMPTOM	CAUSE	FIX
redis restarting, log <code>Can't handle RDB format version 11</code>	another compose project named <code>docker</code> on this machine had a <code>redis</code> service, its volume was reused	<code>docker compose rm -sfv redis &&</code> <code>docker compose up -d</code>
container name <code>"/merginmaps-redis"</code> is already in use	another Mergin stack on the same machine	<code>docker rename merginmaps-redis old_redis</code> or remove that stack
Found orphan containers warning	project name shared with another compose folder	we pass <code>-p laax</code> , warning disappears

Login and database

SYMPTOM	CAUSE	FIX
login returns 500, relation "user" does not exist	<code>flask init</code> not run	<code>docker compose exec server flask init -e ...</code>
tables exist but login fails after a config change	<code>DB_DATABASE</code> changed after init	re-run init, or set it back
lost admin password	random, printed once	<code>docker compose exec server flask user create owner2 <pw> --is-admin --email ...</code>
"Server not properly configured"	<code>MERGIN_BASE_URL</code> still <code>fixme</code>	fix <code>.prod.env</code> , <code>docker compose up -d</code> recreates what changed

The field device cannot reach the server

SYMPTOM	FIX
Windows app: "server not properly configured" or login hangs	<code>MERGIN_BASE_URL</code> must be <code>http://localhost:8080</code> , then <code>docker compose up -d</code>
Windows app cannot connect at all	browser on Windows, <code>http://localhost:8080</code> . Blank: Docker Desktop not publishing, restart it
plugin logs into <code>app.merginmaps.com</code> instead of your server	Configure dialog, the custom server box is unticked in this QGIS profile
project not listed for <code>field</code>	not shared, module 04 last step
Linux app: no window, log says <code>Failed to initialize graphics backend for OpenGL</code>	start it with <code>run.sh</code> , it sets <code>QT_QUICK_BACKEND=software</code>
Linux app: <code>Could not find the Qt platform plugin "wayland"</code>	the build has no Wayland plugin, <code>run.sh</code> forces <code>QT_QPA_PLATFORM=xcb</code>
Linux app: <code>Cannot find proj.db</code>	<code>PROJ_DATA</code> must point to <code>share/input/qgis-data/proj</code> , again <code>run.sh</code>

A real phone, one day

1. `MERGIN_BASE_URL=http://<lan-ip>:8080` in `.prod.env`, then `docker compose up -d`
2. laptop browser on `http://<lan-ip>:8080`, not `localhost`
3. phone browser, same URL. Blank or timeout: network, not Mergin
4. Linux: `sudo ufw allow 8080/tcp`
5. Windows: allow Docker Desktop backend on private networks in Defender Firewall
6. Wi-Fi that isolates clients: a hotspot or a travel router

Useful commands

```
docker compose logs -f server celery-worker  
docker compose exec server flask server check  
docker compose exec server flask server permissions --path /data  
docker compose exec redis redis-cli monitor  
docker compose exec db psql -U postgres -d mergin -c '\dt'
```

After editing `.prod.env`, `docker compose up -d` recreates only the containers whose configuration changed. If the server container was recreated, `docker compose restart proxy` as well.

`diagnostic_logs/` holds detailed server logs when the container output is not enough.

Grafana: start the observability stack

The server repo ships it as an option, in `common/telemetry/`. Same `mergin` Docker network as the server.

```
cd ~/workshop_mm_laax/laax/common/telemetry
docker compose -f docker-compose.otel.yml up -d
```

<http://localhost:3000>, no login.

- Dashboard **Containers**: CPU and memory of every container, refreshed every 10 s
- **Explore**, data source **Loki**, query `{container_name="merginmaps-server"}`: the server log, filterable
- A quiet server logs one `[ACCESS]` line per request, a few `[INFO]` and `[WARNING]`. Nothing more until someone syncs.

Stop it: `docker compose -f docker-compose.otel.yml down`. Logs stay in `loki-data/`.

Grafana: what arrives, what does not

DATA SOURCE	CONTENT WITH IMAGE 2025.7.3	COLLECTED BY
Loki	logs of every container	Alloy, from the Docker socket
Prometheus	container CPU and memory, Redis internals	the OpenTelemetry collector
Tempo	nothing	needs <code>OTEL_ENABLED=1</code> in <code>.prod.env</code> and a backend image newer than 2025.7.3

The OpenTelemetry code landed on the server's `master` in 2026. No Community Edition image has been published since 2025.7.3, so request traces and gunicorn metrics stay out of reach today.

Start over

```
docker compose down -v  
sudo rm -rf mergin_db projects diagnostic_logs
```

Stops the stack and deletes the database and every project. `.prod.env` stays.

Then modules 02 and 03 again: `set_permissions.sh`, `docker compose up -d`, `flask init`.

09 Wrap-up

What you did

- ran a seven-container Mergin Maps CE server from one compose file
- configured it with six lines in `.prod.env`
- created users, pushed a project from QGIS, surveyed as a second user, pulled back
- mirrored the survey GeoPackage into PostGIS with db-sync, both directions
- read logs, fixed permissions, know where the data lives

Stop, keep, or wipe

```
docker compose down                # stop, keep database and projects
docker compose up -d               # same data comes back
docker compose down -v && sudo rm -rf mergin_db projects diagnostic_logs # wipe
```

DB Sync stack, from `~/workshop_mm_laax/laax/dbsync`: `docker compose down` keeps PostGIS data,
`docker compose down -v` wipes it.

From laptop to a real server

TODAY	PRODUCTION
<code>http://localhost:8080</code>	<code>https://mergin.example.ch</code> , TLS on a reverse proxy, see <code>common/ssl-proxy.conf</code>
<code>MAIL_SUPPRESS_SEND=1</code>	real SMTP, invitations and password resets work
data in <code>docker/</code>	dedicated disk, nightly <code>pg_dump</code> and <code>projects/</code> backup
<code>flask init</code> once	upgrade path: pull new images, <code>flask db upgrade</code>
one laptop	8 GB RAM, 2 vCPU minimum, Linux host

Read next

- Install: <https://merginmaps.com/docs/server/install/>
- Secure: <https://merginmaps.com/docs/server/security/>
- Upgrade: <https://merginmaps.com/docs/server/upgrade/>
- Administer: <https://merginmaps.com/docs/server/administer/>
- Troubleshoot: <https://merginmaps.com/docs/server/troubleshoot/>
- Source: <https://github.com/MerginMaps/server>

Bonus: teach Claude Code this setup

A **skill** is one Markdown file. Claude loads it when the request matches its description.

PART	WHAT
Where	<code>.claude/skills/<name>/SKILL.md</code> in the repo, or <code>~/claude/skills/</code> for you only
Frontmatter	<code>name</code> , plus <code>description</code> : what triggers it, so start with "Use when..."
Body	steps, exact commands, the check after each, a failure table
Run it	<code>/mm-community</code> , or just ask "get the Mergin server running"

Iterate: run it on a fresh clone, watch where Claude stumbles, tighten that step.

The example skill, trimmed

```
---
name: mm-community
description: Use when getting the Mergin Maps Community Edition (CE) server
  running on a laptop from this workshop repo, or when it fails to start.
  Triggers on "mergin server", "get the workshop server running", ...
---
## 3. Start
make up
make status
Expect seven containers `Up`. Wait for /ping before init.
## 4. Init, check, users
make init EMAIL=owner@example.com
Show the user the printed password: it appears once.
## Failures
| permission denied on `/data` | `make dirs`, then `make restart` |
```

Questions